

Documentación RobusPredictor

Clase del modelo

```
class robuspredictor.RobusPredictor(  
    n_min,  
    n_max,  
    n_dom,  
    mean_min,  
    mean_max,  
    std_min,  
    std_max,  
    use_default_value=True,  
    default_value=0,  
    verbose=False  
)
```

Parámetros del modelo

n_min : int

Tamaño mínimo permitido para un cubo. El modelo no seguirá dividiendo una región si al hacerlo se generan subconjuntos menores a este valor.

n_max : int

Tamaño máximo permitido para un cubo.

mean_min : float

Promedio mínimo permitido para el target dentro de un cubo. Si el promedio del target en un cubo queda por debajo de este valor en algún dominio, el cubo puede ser considerado zona roja.

mean_max : float

Promedio máximo permitido para el target dentro de un cubo. Si el promedio del target en un cubo supera este valor en algún dominio, el cubo puede ser considerado zona roja.

std_min : float

Desviación estándar mínima permitida para el target dentro de un cubo.

std_max : float

Desviación estándar máxima permitida para el target dentro de un cubo. Si la dispersión del target en un cubo supera este valor en algún dominio, el cubo puede ser considerado zona roja.

n_dom : int

Cantidad de dominios en que se divide el conjunto de entrenamiento. El valor mínimo permitido es `2`, ya que el modelo necesita al menos un dominio para aprender cortes y otro dominio para evaluar estabilidad.

use_default_value : bool, default=True

Define qué ocurre cuando una observación cae en una zona roja durante la predicción.

Si `use_default_value=True`, las zonas rojas reciben el valor definido en `default_value`.

Si `use_default_value=False`, las zonas rojas reciben su propio `prediction_value`.

default_value : int | float, default=0

Valor usado como predicción cuando una observación cae en una zona roja y `use_default_value=True`.

verbose : bool, default=False

Si es `True`, muestra información adicional durante el entrenamiento y la predicción.

Atributos del modelo

Atributo	Tipo	Disponible después	Descripción
----------	------	--------------------	-------------

		de	
<code>domains</code>	<code>list[dict]</code>	<code>fit()</code>	Dominios de entrenamiento generados por el modelo.
<code>stable_cubes</code>	<code>list[dict]</code>	<code>fit()</code>	Cubos que cumplieron las condiciones de estabilidad.
<code>red_zones</code>	<code>list[dict]</code>	<code>fit()</code>	Cubos rechazados por no cumplir estabilidad.
<code>cuts</code>	<code>list[dict]</code>	<code>fit()</code>	Cortes aprendidos por el árbol de particiones.
<code>feature_names</code>	<code>list[str]</code>	<code>fit()</code>	Columnas usadas durante el entrenamiento.
<code>is_fitted_</code>	<code>bool</code>	<code>fit()</code>	Indica si el modelo ya fue entrenado.
<code>last_predictions</code>	<code>pd.Series</code>	<code>predict()</code>	Últimas predicciones generadas por el modelo.
<code>checkpoint</code>	<code>dict[str, pd.DataFrame]</code>	<code>export_checkpoint()</code>	DataFrames usados para exportar el checkpoint.

`domains : list[dict]`

Dominios generados durante el entrenamiento. Cada dominio contiene los datos separados internamente por el modelo y los grupos generados al aplicar los cortes. Este atributo permite inspeccionar cómo se dividió el conjunto de entrenamiento.

`stable_cubes : list[dict]`

Lista de cubos considerados estables. Un cubo estable es una región que cumple las restricciones definidas por `mean_min`, `mean_max`, `std_min` y `std_max` entre los dominios de entrenamiento.

```
{
  "group_id": "LLR",
  "prediction_value": 1.75,
  "stats": [...],
  "is_stable": True
}
```

red_zones : list[dict]

Lista de zonas rojas generadas durante el entrenamiento. Una zona roja corresponde a una región que no cumple las condiciones de estabilidad. Puede deberse a promedio fuera de rango, desviación estándar fuera de rango o falta de consistencia entre dominios.

```
{
  "group_id": "RLL",
  "prediction_value": 2.35,
  "rejection_reasons": [...],
  "stats": [...],
  "is_stable": False
}
```

cuts : list[dict]

Lista de cortes aprendidos durante el particionamiento recursivo. Cada corte representa una división del árbol e incluye información como nodo, variable usada para cortar, rutas izquierda/derecha y valores límite.

```
{
  "node_id": "ROOT",
  "level": 0,
  "variable": "var1",
  "left_path": "L",
  "right_path": "R",
  "left_max": 17,
  "right_min": 50,
  "rule": "var1 <= 17"
}
```

feature_names : list[str]

Lista de variables utilizadas durante el entrenamiento.

is_fitted : bool

Indica si el modelo ya fue entrenado.

last_predictions : pd.Series

Últimas predicciones generadas por el modelo.

checkpoint : dict[str, pd.DataFrame]

Diccionario con los DataFrames generados por export_checkpoint().

```
{
    "cubes_checkpoint": pd.DataFrame,
    "cuts_checkpoint": pd.DataFrame,
    "summary": pd.DataFrame
}
```

Funciones

fit(X, y)

Entrena el modelo RobusPredictor.

```
modelo.fit(X_train, y_train)
```

Parámetros

X : pd.DataFrame

Variables predictoras de entrenamiento.

X : pd.DataFrame

Variable objetivo de entrenamiento.

Retorna

self : RobusPredictor

Retorna la instancia entrenada.

predict(x)

Genera predicciones para nuevos datos.

```
y_pred = modelo.predict(X_valid)
```

Parámetros

x : pd.DataFrame

Datos a predecir. Debe contener las mismas columnas utilizadas durante el entrenamiento.

Retorna

y_pred : pd.Series

Serie de pandas con una predicción por registro. No retorna un ndarray, para conservar el índice original del DataFrame.

export_checkpoint(X_valid, y_valid, file_name="checkpoint_robuspredictor", file_format="xlsx")

Exporta el checkpoint general del modelo. Este export incluye información de cubos, cortes, resumen del entrenamiento y métricas de validación por cubo.

```
checkpoint = modelo.export_checkpoint(  
    X_valid=X_valid,  
    y_valid=y_valid  
)
```

Parámetros

X_valid : pd.DataFrame

Dataset de validación. Se aplican los cortes aprendidos para asignar cada registro al cubo correspondiente.

y_valid : pd.Series

Target real asociado a X_valid. Se utiliza para calcular métricas de validación por cubo.

target real asociado a `X_valid`. Se utiliza para calcular métricas de validación por cubo.

file_name : str, default="checkpoint_robustpredictor"

Nombre base del archivo de salida. No es necesario indicar extensión.

file_format : {"xlsx", "csv"}, default="xlsx"

Formato de salida.

- Si `file_format="xlsx"`, genera un archivo Excel con varias hojas.
- Si `file_format="csv"`, genera un archivo CSV por cada hoja.

Retorna

checkpoint : dict[str, pd.DataFrame]

Diccionario con los DataFrames generados.

```
export_prediction_checkpoint(X_valid, y_valid,  
file_name="scoring_robustpredictor", dato_real=None,  
file_format="xlsx")
```

Exporta el detalle de predicción por registro.

```
scoring_df = modelo.export_prediction_checkpoint(  
    X_valid=X_valid,  
    y_valid=y_valid,  
    dato_real=ARRIENDO_REAL  
)
```

Parámetros

X_valid : pd.DataFrame

Datos de validación o datos a predecir.

y_valid : pd.Series

Target real asociado a `X_valid`.

file_name : str, default="checkpoint_robustpredictor"

Nombre base del archivo de salida. No es necesario indicar extensión.

dato_real : pd.Series, optional

Variable real adicional utilizada para evaluación o comparación.

file_format : {"xlsx", "csv"}, default="xlsx"

Formato de salida.

Retorna

scoring_df : pd.DataFrame

DataFrame con el detalle de predicción por registro.

best_percentage(y_target, top_pct=0.05)

Calcula la precisión del modelo dentro del Top N% de las últimas predicciones generadas.

```
top_5 = modelo.best_percentage(  
    y_target=ARRIENDO_REAL,  
    top_pct=0.05  
)
```

Parámetros

y_target : pd.Series

Variable real contra la cual se evalúa la precisión del Top N%.

top_pct : float, default=0.05

Porcentaje superior de predicciones a evaluar.

Retorna

precision : float

Precisión obtenida dentro del Top N% de predicciones más altas.

predict_cubes(x)

Asigna cada registro al cubo correspondiente del modelo.

```
cube_ids = modelo.predict_cubes(X_valid)
```


Parámetros

x : pd.DataFrame

Datos a asignar a cubos. Debe contener las mismas columnas utilizadas durante el entrenamiento.

Retorna

cube_ids : pd.Series

Serie de pandas con el identificador público del cubo asignado a cada registro.

export_dataframe_cubes()

Retorna un DataFrame resumen de los cubos utilizados en la última predicción

```
cubes_df = modelo.export_dataframe_cubes()
```

Retorna

cube_df : pd.DataFrame

Dataframe con información respecto al ID del cubo, valores mínimos y máximos de cada variables y la predicción correspondiente al cubo.

export_cubes_grid()

Retorna un DataFrame con la grilla final de cubos aprendida por el modelo.

```
cube_grid= modelo.export_cubes_grid()
```

Retorna

cube_grid : pd.DataFrame

Dataframe que contiene el ID del cubo, el grupo al que pertenece según el árbol de cortes, si es estable o no, la predicción del cubo, el conjunto de reglas de cortes por mediana y la forma "cruda" en la que se crea la grilla de cortes